



Article

OpenPCle: An Open-Source PCIe Controller

Somoye Idris, David Jovel and Lamia Mannan



Article

OpenPCIe: An Open-Source PCIe Controller

Somoye Idris , David Jovel  and Lamia Mannan 

Department of Electrical and Computer Engineering, Morgan State University, Baltimore, MD 21251, USA; davaq1@morgan.edu (D.J.); laman14@morgan.edu (L.M.)

* Correspondence: idris.somoye@morgan.edu

Abstract

Peripheral Component Interconnect Express (PCIe) is a critical interface for FPGA-based accelerators, yet existing controller solutions are often proprietary, costly, and/or incompatible with open-source workflows. We present a fully open-source PCIe controller, written in synthesizable Verilog and optimized for Field-Programmable Gate Array (FPGA) deployment. The core is verified using a Python-based cocotb_{2.0.1} and pyuvmm_{4.0.0} testbench with a modeled Root Complex (RC), complete with data packet generation, automated checks for enumeration, flow control, and retry mechanisms. On an AMD Xilinx AC701 (XC7A200T), the design achieves less than 6% LUT utilization, timing closure at 100 MHz user clock, and demonstrates compatibility with vendor transceivers. Reference builds also meet timing on Altera Agilex devices with similar resource utilization. All RTL, verification infrastructures, and example designs are publicly released, enabling reproducible research and accelerating the development of PCIe-enabled systems for high-speed data acquisition, NVMe front-ends, and custom FPGA accelerators.

Keywords: FPGA; PCIe; high-speed-interfaces; AMD Xilinx; open-source; digital circuits

1. Introduction

PCI originated as a shared, parallel, multi-drop local bus intended to provide a standardized processor-peripheral interface with memory-mapped and I/O-mapped transactions. As server and workstation bandwidth demands increased, PCI-X extended the conventional PCI model with higher operating frequencies and protocol improvements (e.g., split transactions) while retaining substantial software and ecosystem compatibility. Even with these improvements, both PCI and PCI-X remained constrained by shared-bus electrical loading and limited scalability, motivating a transition to switched serial links [1,2].

PCIe emerged in the early 2000s as the successor to shared parallel buses such as PCI and PCI-X, replacing them with a point-to-point high-speed serial interconnect that scales bandwidth by aggregating multiple lanes into a link. The interconnect is organized as a packet-switched fabric rooted at a host (the Root Complex (RC)) and extended through switches to multiple endpoints, allowing a system to scale from simple single-device links to complex multi-device topologies while preserving a largely uniform programming model. A defining feature of PCIe is that it preserves the conventional PCI software stack through configuration space, enumeration, and a load/store transaction model (memory, I/O, and configuration transactions), while the underlying wire protocol evolves across generations to increase throughput and improve signal integrity.

Despite its importance, PCIe remains one of the least accessible interfaces within the open-source hardware ecosystem. Commercial endpoint controller solutions, whether



Academic Editor: Douglas O'Shaughnessy

Received: 4 March 2026

Revised: 26 March 2026

Accepted: 29 March 2026

Published: 1 April 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

integrated into FPGA vendor toolchains [3,4] or available as a standalone IP [5,6], are often cost-prohibitive, distributed under restrictive licenses, or tightly coupled to proprietary design flows. This creates significant barriers to entry, particularly for academic institutions, independent developers, and open hardware initiatives that rely on open-source tools for transparency, reproducibility, and collaborative development [7].

The PCIe ecosystem is standardized by PCI-SIG through the Base Specification, which defines the layered architecture and interoperability requirements between vendors [8]. Functionally, PCIe is structured into the Transaction Layer, Data Link Layer, and Physical Layer, with optional capabilities (e.g., power management, error reporting, virtualization, and atomic operations) negotiated through configuration mechanisms and capability structures. In practice, systems are deployed in width/speed combinations (e.g., $\times 1$, $\times 4$, $\times 8$, $\times 16$ at successive generations), and platform implementations typically terminate the Physical Layer in a Serializer/Deserializer (SERDES)/Physical Layer (PHY) (often via a Physical Interface for PCI Express (PIPE)-style interface), while the upper layers are implemented in logic as a controller IP.

PCIe is organized into three primary architectural layers (Figure 1). The Transaction Layer defines the software-visible communication model and packet formats. It generates and consumes Transaction Layer Packets (TLPs) for memory reads/writes, I/O, and configuration. It implements routing attributes and ordering rules, and optionally supports features such as address translation, atomics, and message-based events. The Data Link Layer provides per-link reliability and flow control on top of the Transaction Layer. This is done by adding sequence numbers, Link CRC (LCRC), and acknowledgment/negative-acknowledgment (ACK/NAK) handling with replay; it also carries Data Link Layer Packets (DLLPs) used for link management (e.g., credit-based flow control updates and power-management notifications). The Physical Layer covers electrical signaling and link bring-up, including lane initialization and training, speed/width negotiation, equalization, and the symbol/bit-level encoding and scrambling needed to transmit data over high-speed differential serial lanes; in FPGA-based designs, this layer is typically split between a vendor SERDES/PHY and a standardized interface (commonly PIPE) presented to the controller logic. OpenPCIe is positioned to bridge this gap by providing a more complete, fully open-source implementation of the Endpoint controller that spans from the Transaction Layer through the Data Link Layer and interfaces directly to the Physical Layer transceiver.

This paper details the design, implementation and release of a fully open-source PCIe Endpoint Controller Intellectual Property (IP) core, optimized for deployment on FPGA platforms. PCIe remains a cornerstone of high-speed serial communication in modern embedded and computing systems, providing the bandwidth, scalability, and interoperability required for applications ranging from consumer electronics to high-performance computing [8,9]. Access to a flexible, verifiable and synthesis-ready PCIe option is therefore critical to enable innovation in research, education, and product development.

PCIe is also the dominant host interconnect for modern GPUs and accelerators: discrete GPUs attach to the CPU/RC as PCIe endpoints, using Direct memory access (DMA) to move command buffers and data between system memory and device memory, and relying on interrupt and error-reporting mechanisms defined by the standard. Similarly, FPGA accelerator cards commonly expose one or more PCIe functions (often with vendor- or platform-specific DMA engines) to provide a low-latency path for streaming data, coherent or non-coherent memory access, and control-plane management in heterogeneous systems. While alternative links (e.g., dedicated GPU interconnects) may be used for device-to-device communication, PCIe remains the most ubiquitous and interoperable path for attaching accelerators to general-purpose hosts, making open, verifiable endpoint implementations valuable for research prototypes and open hardware platforms.

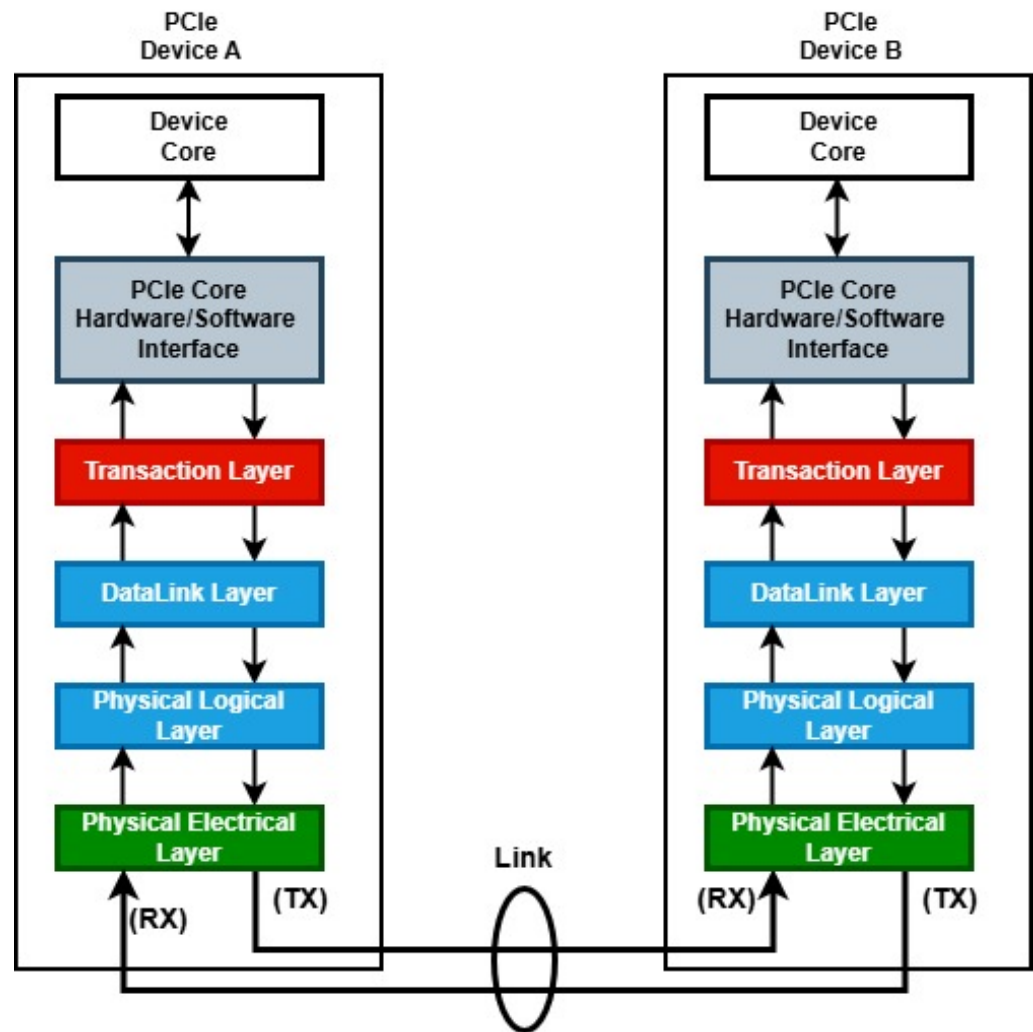


Figure 1. PCIe system packet overview.

In contrast to interfaces such as USB [10] and Ethernet [11]—where mature open-source controller implementations are widely available—PCIe remains comparatively under-served by fully open, standards-complete, and timing-robust implementations. The protocol complexity, strict interoperability behavior, and tight timing requirements across the Transaction Layer, Data Link Layer, and Physical Layer boundaries have limited the availability of open alternatives that are practical for production-style integration. This gap reduces users’ ability to freely modify and experiment with low-level features such as configuration-space behavior and Data Link Layer policy settings.

Our PCIe endpoint IP core directly addresses this gap by providing an implementation that supports Gen1 data rates, fully compliant with the PCIe Base Specification [8]. Written entirely in synthesizable Verilog, the core is structured for straightforward integration into FPGA-based systems, with modular interfaces that accommodate both lightweight embedded designs and high-throughput data pipelines. The design is optimized for resource efficiency to ensure viability on mid- to high-range FPGA platforms while maintaining the throughput and protocol compliance expected of common PCIe solutions.

Verification leverages modern open-source methodologies that combine Python-based cocotb testbenches [12] for co-simulation to ensure protocol correctness and design robustness. This approach facilitates highly modular, reusable, and automated test environments that are both developer-friendly and well-suited to continuous integration pipelines. The

project also uses FuseSoC [13] for IP packaging and distribution, enabling easy dependency management and integration with broader open-source projects System on Chip (SoC).

By releasing a verified, standards-compliant PCIe endpoint IP core, this work seeks to close one of the most significant gaps in the open-source hardware ecosystem, particularly in the rapidly growing RISC-V-based SoC development landscape [14]. In doing so, it will remove a key barrier that has historically constrained the adoption of open-source hardware in performance-critical applications, thereby enabling new opportunities for research, industry prototyping, and community-driven innovation.

The specific contributions of this work include:

- A fully open-source, synthesizable PCIe endpoint controller IP core written in Verilog.
- Integration support for multiple FPGA platforms and open-source SoC projects.
- A comprehensive and automated verification flow using cocotb and Python-based test benches.
- Packaging and distribution through FuseSoC to promote reuse and collaboration.

The remainder of this paper is organized as follows. Section 2 reviews prior efforts and related work on open-source PCIe implementations and verification frameworks. Section 3 provides an overview of our PCIe endpoint controller architecture, including the transmit and receive datapaths. Section 4 details the larger modules that comprise the system, including the PIPE interface and the Data Link Layer. Sections 5 and 6 present the simulation and validation methodology, highlighting the use of cocotb and pyuvvm for verification and the availability of example designs for hardware validation. Section 7 explains the utilization of resources in our design with a common FPGA toolchain. Finally, Sections 8 and 9 summarize our contributions and discuss the directions for future work.

2. Related Work

PCIe has been widely adopted as the de facto interconnect standard for high-speed communication in computing systems [15]. Commercial endpoint controller IP cores are available from FPGA vendors such as AMD Xilinx and Altera [3,4], offering highly optimized implementations with advanced features, including DMA engines for scatter-gather, multifunction endpoints, and support for Gen3/Gen4 data rates. However, these solutions are often distributed under restrictive licenses, tightly coupled to proprietary toolchains, and not accessible for modification or redistribution, thereby limiting their suitability for research and open-source development.

Several open-source initiatives have attempted to address this gap by providing partial or complete PCIe endpoint implementations. For example, LitePCIe [16] provides a Python and Migen-based PCIe core that generates Transaction Layer Packet (TLP)s and offers a software-friendly driver interface. Unfortunately, LitePCIe remains a TLP-level implementation and still depends on FPGA vendor IP to provide the PHY and Data Link Layer Packet (DLLP) functionality, including link training, flow control, and error handling. This reliance on proprietary blocks prevents a truly vendor-neutral solution and limits reproducibility in open-source flows.

We also consider low-level physical-layer explorations. The pcie5_phy project [17] mainly targets Gen5 speeds, but includes a low-level PHY wrapper and associated logic. Despite this, it does not include features such as configuration space, Base Address Register (BAR)s, or Direct memory access (DMA), and it is currently not synthesizable; its value is primarily as a verification, especially for exploring encoding/decoding and link training.

Other contributions have specifically explored the physical and Data Link Layers of PCIe. The work in [18] presents a design and simulation of the Physical Layer of PCIe, focusing on scrambling, start/end framing, and 8b/10b encoding/decoding. Although this provides important insights into physical-layer modeling, it does not extend to higher-layer

functionality or to a deployable endpoint. Similarly, ref. [19] addresses efficient buffering strategies in the Data Link Layer, including flow control and packet management, which are critical for achieving high throughput and reliability. Furthermore, the work in [20] focuses on the detailed design and simulation of the Data Link Layer. This contribution highlights the importance of sequencing, LCRC generation, and retry logic, but, as with many academic efforts, it does not provide a fully synthesized endpoint controller. However, both papers demonstrate the need for open-source cores that integrate all PCIe layers into a cohesive, deployable implementation. This work highlights the complexity of DLLP implementations and underscores why most open PCIe projects avoid implementing this layer in full.

Projects within the OpenCores ecosystem [21] provide Transaction Layer logic and some abstraction modules, but do not implement the Physical Layer or Data Link Layer. As a result, these designs cannot independently establish a PCIe link, perform link training, or provide the error-detection and retry mechanisms required for compliance. Moreover, many of these projects are not actively maintained and lack support for recent revisions of the PCIe specification, further limiting their applicability in modern FPGA and SoC design flows.

Recent academic open-source hardware efforts also highlight this gap. For example, the openPCIe project [22] promotes open-source hardware design, yet must rely on closed-source PCIe IP due to the limited availability of fully open, standards-compliant PCIe building blocks. This work helps address that limitation by providing an open-endpoint controller option that can be integrated into such platforms, thereby reducing reliance on proprietary IP and enabling more transparent and reproducible accelerator designs.

To summarize prior projects in a structured way, Table 1 compares their strengths and limitations, and maps each limitation to the corresponding design decision in this work.

Table 1. Related-work comparison: strengths, weaknesses, and how OpenPCIe design decisions address the weaknesses.

Project (Reference)	Data Link + Physical Layer Implemented (Open Source)	Strong Sides	Weak Sides	How OpenPCIe Addresses Weaknesses
OpenPCIe (this work)	Yes	Open, synthesizable endpoint with TL+DLL logic and integration-ready flow	Current release focuses on Gen1 and a non-DMA baseline path	Roadmap extends to Gen2, broader wrappers, and deeper compliance/performance validation while preserving an open, reproducible implementation.
Vendor PCIe endpoint IP ([3,4])	No (closed implementation)	Mature implementations, high-speed support (Gen3/Gen4), integrated DMA features	Restrictive licensing and tight coupling to vendor toolchains limit transparency and portability	OpenPCIe provides a transparent and modifiable alternative with documented build/simulation flow, reducing dependence on proprietary licenses.
LitePCIe ([16])	No	Practical software-facing stack and active open ecosystem adoption	Mostly TLP-level in open logic; relies on vendor blocks for key PHY/DLL behavior	OpenPCIe implements both TL and DLL in soft logic with a clear PIPE-style transceiver boundary, improving vendor neutrality.
pcie5_phy ([17])	No	Valuable low-level exploration toward high-speed PHY behavior	Not a complete, synthesizable endpoint and lacks system-level endpoint features (e.g., configuration space/BAR)	OpenPCIe provides an end-to-end synthesizable endpoint architecture with configuration handling and integration-ready interfaces.
Academic PHY/DLL prototypes ([18–20])	No	Strong depth on specific mechanisms (scrambling, framing, buffering, retry logic)	Layer-specific focus; no complete, reusable endpoint core for system integration	OpenPCIe combines these mechanisms into one cohesive controller architecture suitable for practical system integration.
OpenCores PCIe efforts ([21])	No	Open availability of partial TL logic and educational value	Incomplete layer coverage, limited maintenance, and reduced applicability to modern flows	OpenPCIe prioritizes maintainable modularization, full protocol-path coverage, and documentation for current FPGA/SoC integration.

In addition to hardware efforts, open source verification methodologies have gained traction. Frameworks such as cocotb [12] and pyuvvm [23] enable Python-driven cosimulation-based test benches [24], which improve modularity and reduce development overhead compared to traditional SystemVerilog/UVM approaches [24]. These tools have been successfully applied in verifying complex SoC interconnects [12], demonstrating their viability for industry-grade verification flows.

Despite these contributions, there remains no fully open, vendor-neutral PCIe endpoint controller IP that is protocol-compliant, resource-efficient, and verified using modern open-source methodologies. This work aims to fill this gap by providing a synthesizable, formally verified, and cocotb-tested PCIe Gen1 endpoint core that can be easily integrated into a wide range of FPGA-based systems and open-source SoC projects.

3. System Overview

Our system is a PCIe Endpoint Controller IP core designed for FPGA-based deployment. The architecture adheres to the PCIe Base Specification, which encompasses the Transaction Layer, Data Link Layer, and Physical Layer, with each layer implemented in synthesized Verilog to ensure portability and FPGA compatibility. Figure 2 illustrates the high-level architecture of the system. The Physical Layer transmit and receive blocks are highlighted in blue. This layer handles 8b/10b encoding/decoding for Gen1/2 link speeds, lane management, and coordination with the link training and status sequence (LTSSM), which is colored purple. It interfaces to a vendor transceiver through a PIPE-style boundary and supports integration with an external SERDES wrapper.

The Transaction Layer manages the formatting, addressing, and ordering rules of the packet. The Data Link Layer ensures reliable delivery of Transaction Layer Packet (TLP)s by providing sequence numbering, acknowledgment (ACK/NAK) handling, and generation and verification of Cyclic Redundancy Check (CRC)s. Above this, the Transaction Layer assembles and routes requests and completions, mediates access to configuration space (including BAR discovery), and exposes a clean interface to user logic for DMA engines, register files, and streaming datapaths.

The endpoint controller integrates with the user logic via an AXI4-Stream interface [25], enabling high-throughput communication between the FPGA fabric and the PCIe link. This modular interface design enables rapid adaptation to various system-on-chip (SoC) environments and custom accelerators. The internal packet movement and interface handoff are illustrated in Figure 3. To support interoperability, the controller has been validated with multiple commercial RCs across different motherboard vendors and operating systems.

A configuration and control module manages PCIe enumeration, BAR assignment, and access to the configuration space, ensuring compatibility with standard PCIe discovery protocols. Furthermore, the design supports multiple independent BARs with configurable size and type, allowing flexible memory-mapped communication between the host and the endpoint.

From a verification perspective, the system employs a hybrid verification strategy that combines cocotb-based Python simulation for functional validation and open-source formal verification for protocol compliance. Hardware validation is performed on representative FPGA platforms (AMD Xilinx AC701).

Optimization for FPGA deployment has been a central design goal. The IP core incorporates pipelined data paths to sustain line-rate throughput while minimizing logic utilization. Clock domain crossing (CDC) modules are used between the PHY interface and the FPGA user logic to ensure reliable operation across asynchronous domains without sacrificing performance.

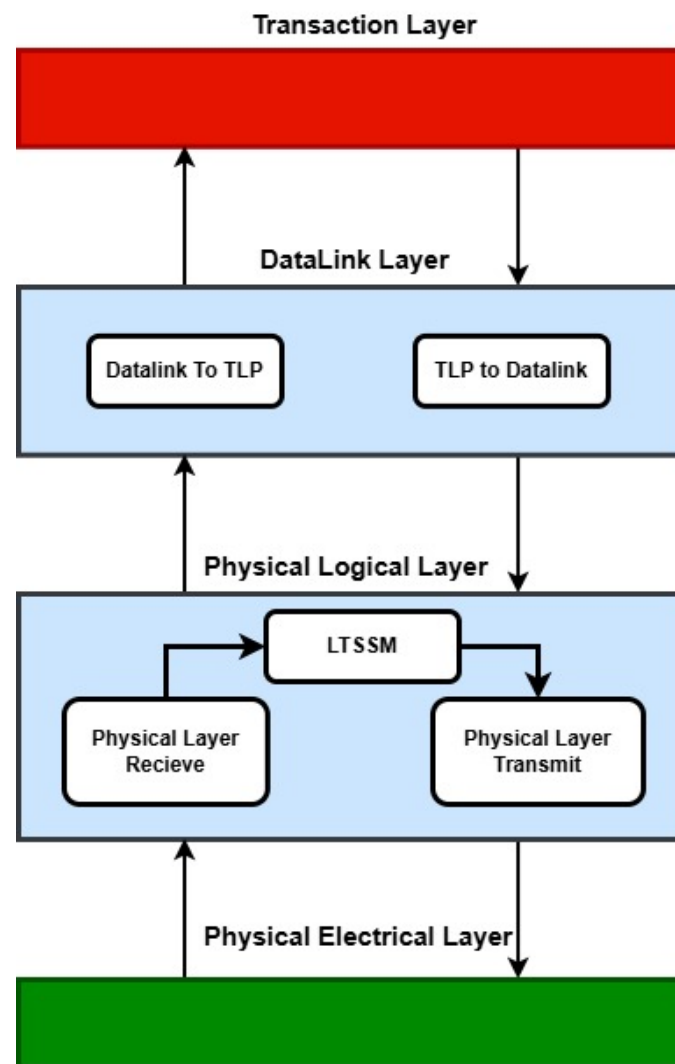


Figure 2. Overall system overview. Blue blocks denote PHY transmit/receive, and datalink functions that OpenPCIe implements, and arrows indicate data/control flow direction.

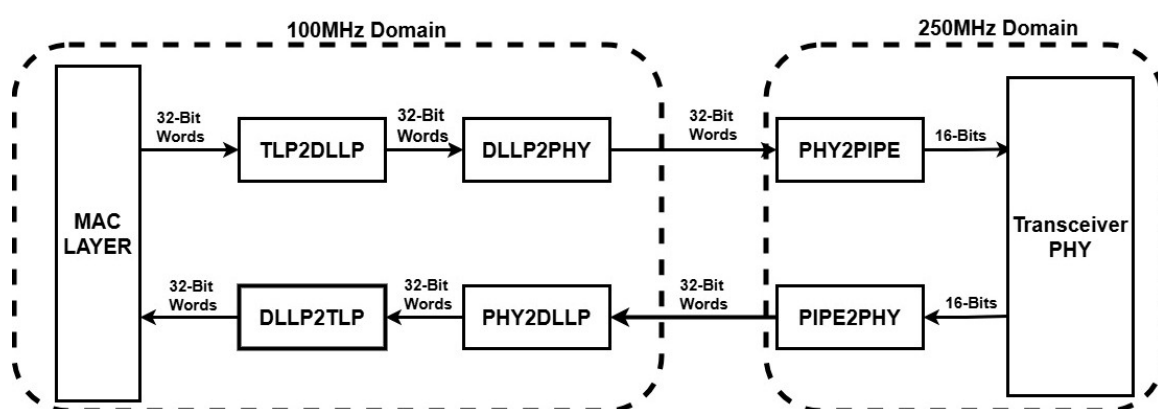


Figure 3. OpenPCIe dataflow between user logic, transaction processing, and the PCIe link interface.

The open-source release includes synthesized RTL, detailed documentation, simulation environments, and example integrations. This enables researchers, educators, and developers to adopt, modify, and extend the PCIe Endpoint Controller for diverse applications, including high-performance data acquisition, storage acceleration, and custom computational pipelines.

4. Design Blocks

4.1. PIPE Interface Architecture

The PIPE serves as the standardized boundary between the PHY and the Media Access Control (MAC) logic, facilitating interoperability between FPGA logic and the high-speed transceiver blocks. In our PCIe Endpoint Controller, Figure 4, the PIPE interface is implemented in accordance with the PIPE 4.4.1 specification [26] and represents a clear line of separation between the transceiver analog front-end and the physical logical layer.

At its lower end, the PIPE interface connects directly to the FPGA's high-speed serial transceiver primitives (for example, AMD Xilinx GTX / GTP or Altera H-Tile), which handle serialization/deserialization (SERDES), 8b/10b encoding and decoding for Gen1/Gen2 operation, and lane-level signal integrity features. The transceiver is expected to present data in parallel with the PIPE interface, along with status and control signals for link initialization, training sequences, and error detection.

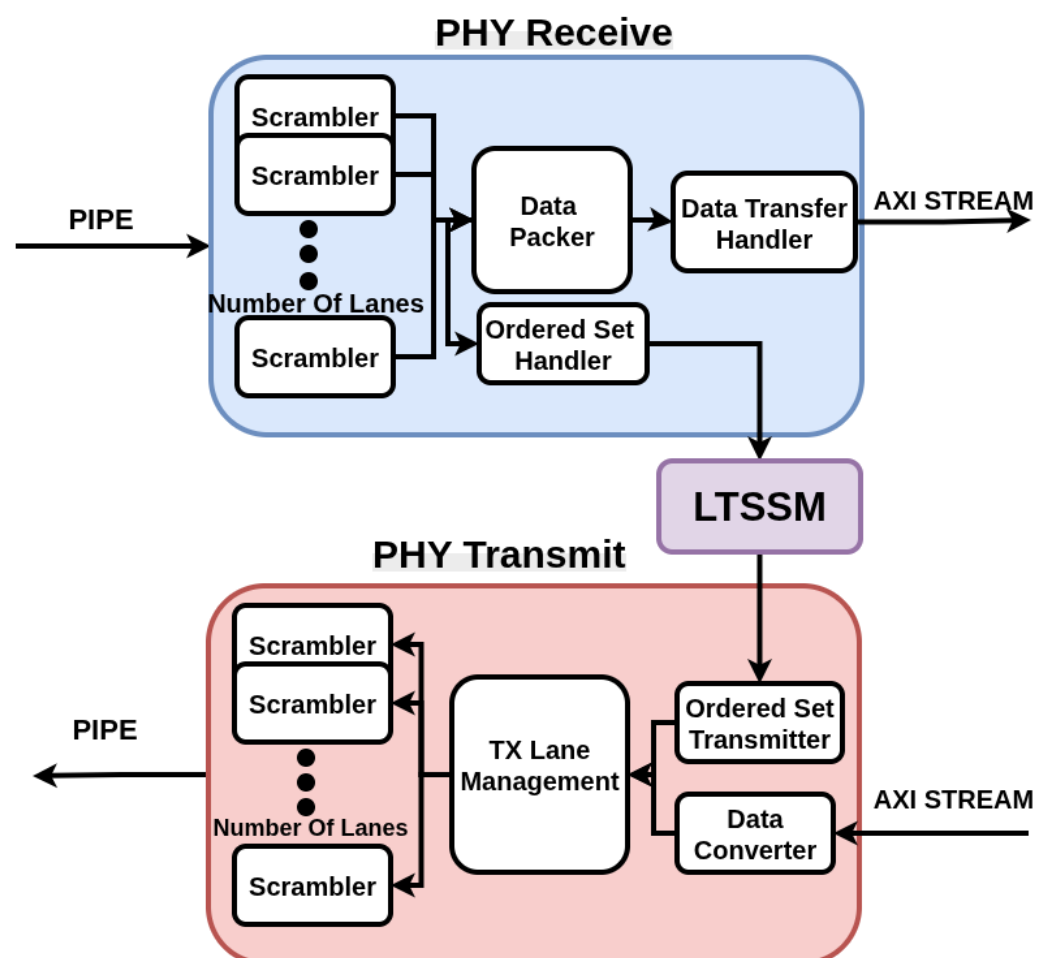


Figure 4. PIPE Interface. Blue denotes the PHY receive module, pink denotes the PHY transmit module, the LTSSM is shown in purple between them, and arrows indicate data flow as well as format and bus type.

Upstream, the PIPE interface interacts with the Data Link Layer (DLL) through a well-defined parallel bus. This interface transmits and receives 8-bit or 16-bit parallel symbols per clock cycle, depending on the negotiated link width and speed. It also propagates critical sideband signals such as:

- Link Training and Status State Machine (LTSSM) Signals—Indicating the current phase of link activation, training, and recovery.

- Data Valid and Data K-Character Indicators—Identifying symbol boundaries and control character locations.
- Error Detection Flags—Reporting Physical Layer errors, including disparity errors, invalid symbols, and overflow /underflow of elastic buffer.

This architectural separation via the PIPE interface offers several key benefits.

1. Physical Layer Abstraction: The endpoint controller logic can be ported across different FPGA platforms and transceiver technologies without modifying higher protocol layers.
2. Design Modularity: Enhancements to the Data Link Layer or Transaction Layer can be implemented independently of PHY hardware changes.
3. Test and Debug Accessibility: The PIPE boundary provides a convenient tap point for simulation, protocol analysis, and hardware debugging using PCIe analyzers.

In our implementation, the PIPE interface runs at 250 MHz, while the internal user interface (i.e., the data-link side) runs at 100 MHz to keep logic design more manageable. Therefore, dedicated clock-domain-crossing (CDC) modules are used between these domains. By adhering to the PIPE specification, the endpoint controller maintains compatibility with both integrated FPGA transceivers and discrete PCIe PHY devices, improving portability and reducing integration complexity.

4.2. Scrambling

The scrambler design in this work builds on previous open-source efforts, such as the PCIe_Scrambler repository [27], which implements Verilog-based scrambling logic for Gen1/2/3 and supports PIPE-compatible scrambler enable/disable features. We re-designed the data pipeline to support higher clock frequencies and implemented more complex disable and bypass logic to accommodate all special-K symbols; these operational states are summarized in Table 2.

The PCIe Physical Layer operates on 32-bit data paths, including within the scrambler. The design uses four byte-wide Linear Feedback Shift Registers (LFSRs) daisy-chained together, advancing by selecting the most significant active byte. This ensures proper whitening across the full data width.

Control logic determines when scrambling is enabled or disabled, particularly during link training sequences, idle states, or compliance patterns. In these cases, the scrambler may freeze, bypass scrambling, or resume advancement to maintain protocol compliance.

Although the LFSR implementation itself is straightforward, the complexity arises from handling many conditional states defined in the PCIe specification. Robust control ensures interoperability, prevents link loss, and enables compliance with standard test suites.

Table 2. Summary of Scrambler Operational States.

State	Description
Advance	The scrambler LFSRs actively shift on each valid data beat. Data is XORed with the scrambler output, ensuring randomized transmission patterns to minimize EMI and preserve DC balance.
Disable	The scrambler LFSRs are held in place, preventing state changes. This mode is used during link training sequences or when deterministic patterns are required by the protocol.
Skip	The scrambler logic is bypassed, and transmitted data flows unaltered. This mode is typically used for diagnostics or cases where scrambling is not mandated.

4.3. Data Link Layer Architecture

The Data Link Layer (DLL) Figure 5, serves as the intermediary between the Physical Layer, accessed through the PIPE interface, and the Transaction Layer, which operates on TLPs. Its primary function is to ensure reliable and ordered packet delivery across an PCIe link by adding sequencing, integrity verification, and flow-control mechanisms to raw data streams.

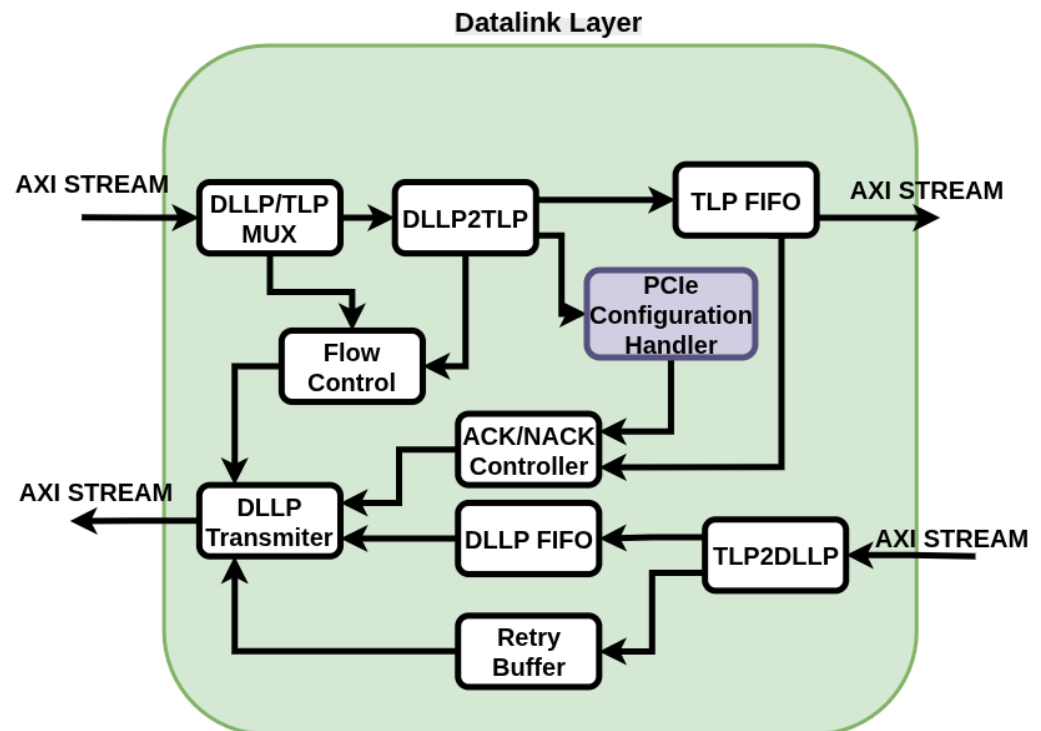


Figure 5. Datalink layer functional modules. Blocks represent functional modules, and arrows represent data and control flow.

On the receiving path, DLL ingests 8-bit or 16-bit parallel symbols from the PIPE interface. These symbols are decoded and assembled into complete Data Link Layer Packet (DLLP)s and Transaction Layer Packet (TLP)s. For received TLPs, the DLL performs the following operations:

- Sequence Number Verification: Ensures that packets are received in the correct order and without duplication.
- Link Cyclic Redundancy Check (LCRC) Validation: Checks the integrity of each packet using the 32-bit LCRC field appended at the end of the frame.
- Acknowledgment/Negative Acknowledgment (ACK/NAK) Generation: Based on the result of sequence and LCRC checks, the DLL issues either an ACK or a NAK DLLP back to the link partner.

On the transmit path, the DLL receives TLPs from the Transaction Layer. Before forwarding these to the PIPE interface, the DLL:

- Assigns a monotonically increasing sequence number to each TLP for ordering and duplicate detection.
- Appends the LCRC field, computed over the entire TLP payload and header.
- Generates any necessary DLLPs, such as Flow Control Updates or ACK/NAK responses, which may be interleaved with outgoing TLPs.

The DLL is also responsible for the retry logic, whereby TLPs that have been negatively acknowledged (NAKed) or not acknowledged within a timeout window are retransmitted

in their original order. This mechanism ensures reliable delivery in the presence of transient link errors without involving the higher protocol layers.

From an implementation standpoint, the PCIe Endpoint Controller incorporates the DLL as a parameterizable RTL module. This module interfaces with the downstream PIPE interface and the MAC/Transaction Layer via the parallel AXI4-Stream handshake protocol.

4.4. Configuration Space Generation with SystemRDL and PeakRDL

The PCIe controller requires a standards-compliant configuration space that provides the host system with critical information, including device identification, capability registers, BAR, and status/control fields. Manually maintaining these register definitions is error-prone, especially as the design evolves or additional features are introduced. To ensure correctness, reproducibility, and portability, the configuration space in this project, highlighted in purple on Figure 5, is specified using the Register Description Language (SystemRDL) standard [28].

SystemRDL enables a high-level, declarative specification of register fields, hierarchies, and access rules. This abstraction allows designers to express the intent of each register (e.g., read-only, write-one-to-clear, volatile) while decoupling the specification from a particular hardware implementation. For the PCIe controller, the RDL files define all standard configuration registers required by the PCIe Base Specification, as well as project-specific control and status registers.

To generate synthesizable Verilog, documentation, and software accessible header files from the SystemRDL specification, we use the open-source PeakRDL toolchain [29]. PeakRDL automatically transforms the RDL description into:

- Verilog RTL modules: Implementing registers with correct reset values, access policies, and update logic.
- C header files: Providing macros and offsets for driver development, ensuring hardware/software consistency.
- HTML/PDF documentation: Capturing register maps in a human-readable format, useful for verification engineers and system integrators.

This flow ensures a single source of truth for the configuration space, reducing the risk of inconsistencies between hardware and software. Furthermore, because PeakRDL is fully open-source, the methodology aligns with the project's open and reproducible nature. The result is a maintainable and transparent configuration space infrastructure that can be easily extended to support additional PCIe capabilities (e.g., MSI-X, advanced error reporting) in future revisions.

5. Simulation

In line with the open source nature of this work, the verification environment for the PCIe Endpoint Controller IP was developed using the cocotb co-simulation framework, the open-source cocotb-pcie library, and the Python-based verification methodology pyuvmm, as shown in Figure 6. This combination ensures that the entire simulation and verification flow remains transparent, reproducible, and accessible to the research and development community. Using both cocotb and pyuvmm, the verification environment benefits from Python's flexibility while adopting the well-known Universal Verification Methodology (UVM) structure [30] for scalability and reusability.

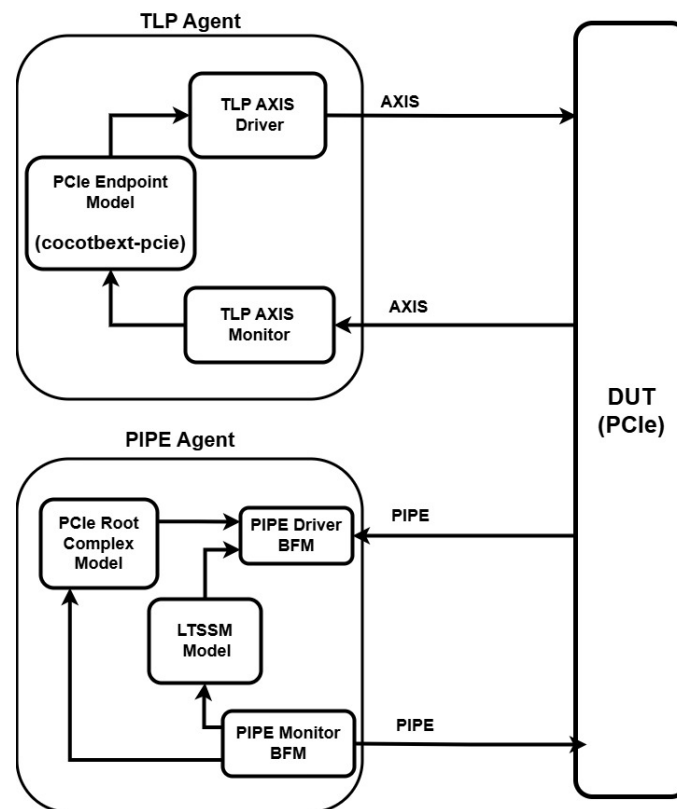


Figure 6. Verification configuration for cocotb simulation.

5.1. Testbench Architecture

The simulation environment consists of three primary components:

1. **Root Complex (RC) Model:** The RC model, implemented using `cocotbext-pcie` [31], is responsible for initiating PCIe configuration transactions, memory reads/writes, and other TLPs. It emulates a host system capable of enumerating and interacting with the endpoint Device Under Test (DUT).
2. **Device Under Test:** The DUT is the PCIe Endpoint Controller IP developed in this project. It is instantiated in the simulation environment as synthesizable Verilog RTL and serves as the bridge between the simulated PIPE interface and the Transaction Layer logic.
3. **Endpoint Functionality Model (EP):** A lightweight endpoint model is implemented to generate application-level data and respond to incoming TLPs when required, allowing end-to-end data path verification between the RC and DUT.

5.2. pyuvvm-Based Verification Components

To bring structure and scalability to the verification process, `pyuvvm` was used to create a UVM-like environment entirely in Python. The following elements were implemented:

- **Sequencers and Sequences:** Python-based sequences were written to generate directed and randomized TLP transactions, covering configuration, memory, and I/O space operations.
- **Drivers and Monitors:** Drivers interact with the DUT through the `cocotb-pcie` bus functional models, while monitors collect observed transactions and forward them to the scoreboard.
- **Scoreboard and Coverage:** A scoreboard compares expected results with observed results to ensure functional correctness. Functional coverage metrics were added to track access to the configuration space, TLP types, and replay buffer activity.

- **Reusable Environment:** The pyuvmm environment is modular and reusable, allowing the same verification environment to be applied to future PCIe core iterations or different endpoint configurations.

5.3. Simulation Flow

The testbench orchestrates the following steps:

1. **Link Training and Enumeration:** The cocotb-pcie RC initiates link training and performs enumeration of the DUT, verifying correct responses to configuration reads/writes and capability structure decoding.
2. **Directed and Random Transactions:** pyuvmm sequences perform memory and I/O operations, verifying packet formatting, completion generation, and correct address decoding.
3. **Stress Testing and Coverage Closure:** Randomized traffic and bursts are applied until functional coverage goals are met, validating robustness under high-load conditions.
4. **Error Injection:** Protocol-level errors such as malformed TLPs, corrupted LCRC fields, and replay buffer overflows are introduced, verifying the DUT's adherence to PCIe error-handling requirements.

6. Experimental Validation

To evaluate the correctness and practical usability of our design, we integrated the core into an existing FPGA reference design by replacing the vendor-provided PCIe IP. Specifically, the standard hard AMD Xilinx PCIe endpoint IP block within the Vivado design example was replaced with our endpoint controller. This approach ensured that all peripheral connections, user application logic, and board-level infrastructure remained unchanged, isolating the PCIe controller as the only variable.

The modified design was then synthesized and implemented using the same timing constraints, clocking scheme, and board configuration as the vendor example. We exercised the design using the vendor-provided host software and existing loopback tests to confirm enumeration, configuration-space access, and basic data transfers. Maintaining the original test environment allowed a direct functional comparison while minimizing confounding changes in the surrounding system. As shown in Figure 7, the vendor AMD Xilinx PCIe IP is highlighted in blue, the OpenPCIe core is highlighted in green, and the Host PC is highlighted in pink.

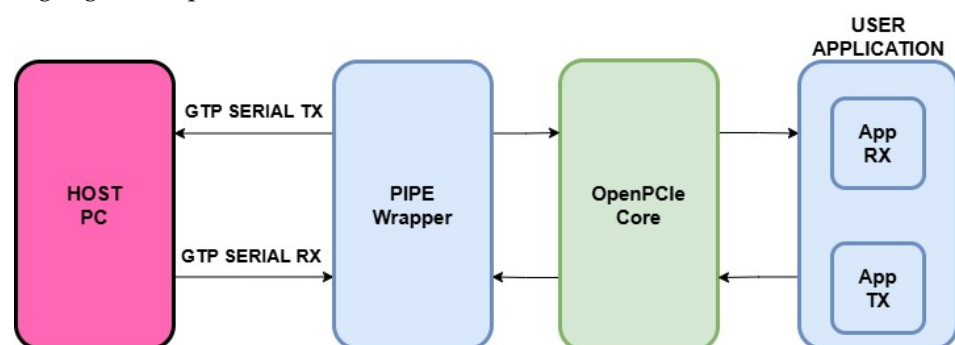


Figure 7. Validation overview of the OpenPCIe endpoint integrated into the vendor reference design.

We further validated link behavior using the 2500A PCIe Protocol Analyzer from ITIC [32]. The analyzer was connected in-line to observe link training, configuration transactions, and basic data movement, providing independent confirmation that the endpoint conforms to expected PCIe traffic patterns.

The FPGA successfully enumerated the custom PCIe endpoint controller when attached to a Linux RC. The Linux kernel correctly identified the device and mapped the BARs exposed by the core, confirming proper operation of the configuration space and

link-layer negotiation. Resource utilization results, presented in Section 7, demonstrate that the endpoint remains lightweight and viable for mid-range FPGA implementations. However, enumeration alone does not validate end-to-end performance.

In simulation using the `cocotb-pcie` framework, the core demonstrated functional completion of Memory Write and Memory Read TLPs, with an average latency of approximately 180–250 ns per transaction at Gen1 speeds, depending on payload size and DLLP acknowledgment timing. Throughput in simulation, while limited by the absence of a DMA engine, reached approximately 110–120 MB/s for continuous BAR-based transfers—well below the theoretical 250 MB/s ceiling of a Gen1 $\times 1$ link.

To fully evaluate achievable bandwidth and latency under realistic workloads, future work will integrate a lightweight DMA controller to enable high-throughput streaming between host memory and the FPGA fabric. This will allow benchmarking of sustained throughput, host–device synchronization efficiency, and interrupt-driven communication using MSI/MSI-X. Additionally, cycle-accurate simulation will be extended to analyze credit consumption, replay-buffer behavior, and retry latency under error-injection scenarios. These enhancements are essential to transition from functional validation toward performance parity with proprietary PCIe solutions.

7. Resource Utilization

The OpenPCIe PCIe endpoint controller was synthesized and implemented on the AMD Xilinx Artix-7 AC701 platform (device: XC7A200T2FBG676C) using Vivado 2022.1. This version was selected for reproducibility and compatibility with the baseline reference design and board support used in this study. The design achieved a successful location and route, and Table 3 summarizes the utilization of the post-implementation resources. To provide context for the efficiency of the proposed OpenPCIe endpoint core, a comparison is made against the LitePCIe core synthesized on an AMD Xilinx 7-Series FPGA using Vivado. LitePCIe, part of the LiteX ecosystem, is among the most mature open-source PCIe implementations.

In general, the OpenPCIe design exhibits low utilization of resources, consuming less than 6% of the available LUTs and less than 3% of the registers, with negligible demand for BRAM, DSPs slices, and clocking resources. Only a single transceiver is required to enable Gen1/Gen2 PCIe operation, making the endpoint lightweight and suitable for integration alongside accelerators or user logic on midrange FPGAs. Direct utilization comparisons should be interpreted with caution, since Gen1 PCIe controllers for the Artix family are typically delivered as hard IP blocks built into silicon and do not consume fabric resources in the same way as a soft PCIe implementation.

Table 3. Resource utilization summary for the PCIe Endpoint Controller on AC701 (XC7A200T-2FBG676C).

Resource	OpenPCIe Used	LitePCIe Used	Available	Utilization Open-PCIe/LitePCIe (%)
LUTs	6303	10,648	133,800	5.52/7.95
Registers	6008	6888	267,600	2.10/2.57
BRAM	4	3	365	1.10/<1
DSPs Slices	0	N/A	740	0.00
Bonded I/O	3	7	400	<1/<1
BUFGCE/BUFGCTRL	10	11	32	31.3/34.3
MMCM	2	2	10	20/20
GTPs Transceivers	1	4	8	12.50/50

To evaluate vendor portability and resource efficiency, the OpenPCIe endpoint controller was synthesized and implemented on an Altera Agilex 5 FPGA using Quartus

Prime Pro Edition 25.3. The target device was A5ED065BB32AE6SR0. The design compiled successfully, demonstrating compatibility beyond AMD Xilinx platforms and highlighting the core's vendor-agnostic architecture.

However, note that this Agilex implementation includes only the OpenPCIe core's logic. It does not include a native PHY interface or transceiver wrapper for Altera's Agilex transceiver architecture (H-Tile/F-Tile). As a result, although the core can be synthesized and placed on the FPGA fabric, it cannot yet form a usable PCIe link without an accompanying PIPE-compliant physical-layer wrapper. The development of this Altera-specific PIPE-to-transceiver bridge is planned as future work.

Table 4 summarizes the post-synthesis resource utilization. The core occupies only 2% of ALMs, indicating minimal logic overhead. No DSPs blocks, HSSI PCIe hard blocks, or transceivers were used, confirming that the design implements the Transaction and Data Link layers in soft logic and interfaces via a vendor-neutral PIPE structure.

Table 4. Resource utilization summary of OpenPCIe on Altera Agilex 5 (A5ED065BB32AE6SR0).

Resource	Used	Available	Utilization (%)
ALMs	4686	222,400	2%
Logic Registers	5265	—	—
Block Memory Bits	13,848	32,993,280	<1%
RAM Blocks	6	1611	<1%
DSPs Blocks	0	846	0%
GTS Transceiver Channels	0	24	0%
HSSI PCIe Hard IP	0	6	0%
PLLs	0	15	0%

8. Discussion

The implementation of a fully open-source PCIe Gen1 endpoint controller represents an important contribution to the open hardware ecosystem, which has seen rapid growth in recent years driven by the proliferation of RISC-V processors and open-source SoC projects such as RocketChip, OpenPiton, and OpenTitan [33,34]. Although open-source cores for CPUs, memory controllers, and other peripheral interfaces have become increasingly mature [35,36], PCIe IP has remained largely proprietary, cost-prohibitive, or locked behind restrictive licensing models. This lack of accessible PCIe solutions has limited researchers' and educators' ability to experiment with high-speed I/O and to prototype heterogeneous systems.

Our work addresses this gap by presenting a synthesizable, standards-compliant PCIe endpoint suitable for deployment on mid-range FPGA platforms such as the Artix AC701. On AC701, the design uses 6303 LUTs (5.52%), 6008 registers (2.10%), 4 BRAMs (1.10%), and 1 GTP transceiver (12.5%), while meeting timing closure for Gen1 operation (2.5 GT/s). For context, the corresponding LitePCIe synthesis in our comparison uses 10,648 LUTs and 4 transceivers on the same family, highlighting OpenPCIe's lower logic and transceiver footprint in this setup. The inclusion of robust clock-domain crossing, flow-control mechanisms, and transaction ordering supports compliance with the PCIe Base Specification and reliable interoperability with commercial RCs.

To make the comparison more explicit, Table 5 summarizes quantitative and integration-focused differences using only values reported in this manuscript. In particular, it contrasts soft-logic protocol-layer coverage, implementation cost, and verification/ecosystem signals relevant to reproducibility.

Table 5. Quantitative and integration-focused comparison between OpenPCIe and LitePCIe (based on values reported in this manuscript).

Comparison Axis	OpenPCIe (This Work)	LitePCIe (As Discussed Here)
Soft-logic layer scope	TL+DLL in soft logic, with PIPE boundary to transceiver (2/3 layers in open logic)	Primarily TLP-level generation; depends on vendor IP for PHY/DLLP behavior (about 1/3 layers in open logic)
LUTs on 7-Series comparison	6303 (Table 3)	10,648 (Table 3)
Transceiver usage (same comparison)	1 GTP	4 transceivers
Simulated non-DMA performance reported in this manuscript	180–250 ns average transaction latency; 110–120 MB/s continuous BAR transfers	Not reported in this manuscript (N/R)
Open verification/tooling integration	cocotb + pyuvvm + FuseSoC flow documented in this work	LiteX/Migen ecosystem usage discussed; equivalent metrics not quantified here

A major focus of this work has been the adoption of modern open-source design, implementation, and verification methodologies. The use of cocotb enables Python-based, coroutine-driven verification, facilitating the creation of reusable, extensible testbenches that can be integrated into continuous integration (CI) pipelines. This aligns with best practices for open-source hardware development [37–39] and enables rapid regression testing across contributors and platforms [7,40].

The open release of this core on GitHub (https://github.com/isomoye-msu/pcie_datalink_layer, accessed on 26 March 2026) along with detailed documentation, simulation models, and FuseSoC integration, creates a platform for collaborative development. Quantitatively, the current verification flow reports functional transaction completion with approximately 180–250 ns average simulated latency and about 110–120 MB/s continuous BAR-transfer throughput without a DMA engine, providing a reproducible baseline for future performance scaling. Not only does it empower researchers to explore PCIe-based accelerators, NVMe-capable designs, and packet sniffers, but it also provides educators with a pedagogical example of high-speed digital interface design, covering topics such as scramblers, CRC generation, pipelining, and retry logic.

Future work will expand this contribution by adding support for PCIe Gen2 (5.0 GT/s) speeds, improving the configurability for different FPGA targets, and extending the verification infrastructure to include formal property checks and hardware-in-the-loop compliance testing. Furthermore, comparison with other available solutions, such as LitePCIe [16], will provide quantitative insight into the trade-offs among latency, throughput, and resource efficiency.

9. Conclusions

This work provides an open-source PCIe Gen1 endpoint IP core, released under a permissive license and hosted on GitHub. The release includes synthesizable Verilog source code, simulation models, implementation scripts, and reference designs targeting platforms such as the AMD Xilinx AC701 and Altera Agilex Apollo. Across the demonstrated targets, the implementation remains lightweight (e.g., 5.52% LUTs on AC701 and 2% ALM utilization on Agilex 5) while maintaining protocol functionality and timing closure at Gen1 operation. By providing a transparent, extensible, and verifiable solution, this project reduces the barrier to adoption and enables FPGA designers to implement high-speed host interfaces for applications ranging from data acquisition and custom accelerators to NVMe storage and PCIe traffic analysis. In addition, we validated integration by replacing vendor PCIe IP in an existing reference design while preserving the original board infrastructure and host software flow. The resulting implementation demonstrates that a fully open

controller can interoperate with standard toolchains and system environments without requiring proprietary hard IP.

Looking ahead, we will extend the core to Gen2 speeds, broaden configurability to support additional FPGA families, and complete vendor-specific PIPE and transceiver wrappers to enable end-to-end hardware link validation. Our near-term measurement targets are explicit: moving beyond the current 110–120 MB/s non-DMA baseline toward sustained host–device throughput characterization, with latency and replay-behavior reporting under controlled traffic and error-injection scenarios.

Author Contributions: Conceptualization, S.I. and D.J.; methodology, S.I., D.J. and L.M.; software, S.I. and D.J.; validation, S.I., D.J. and L.M.; formal analysis, S.I. and D.J.; investigation, S.I., D.J. and L.M.; resources, L.M.; data curation, S.I.; writing—original draft preparation, S.I.; writing—review and editing, D.J. and L.M.; visualization, S.I.; supervision, L.M.; project administration, S.I. and L.M. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Acknowledgments: We acknowledge the CEAMLS Center at Morgan State University for providing hardware resources, and the CAP Center at Morgan State University for the computing resources needed to develop code and run large-scale simulations.

Conflicts of Interest: The authors declare no conflicts of interest.

References

- Sharma, R.; Abeyratne, A. PCI-Express: Evolution of a Ubiquitous Load-Store Interconnect Over Two Decades. *IEEE Circuits Syst. Mag.* **2024**, *24*, 72–95. [CrossRef]
- PCI Local Bus Architecture and Evolution. 1994. Available online: https://ics.uci.edu/~iharris/ics216/pci/PCI_22.pdf (accessed on 26 March 2026).
- Xilinx Inc. *LogiCORE IP PCI Express Integrated Block for UltraScale+ Devices v4.1*; Xilinx Inc.: San Jose, CA, USA, 2021.
- Intel Corporation. *PCI Express Avalon-ST Interface for Endpoints User Guide*; Intel Corporation: Santa Clara, CA, USA, 2022.
- Synopsys Inc. DesignWare PCI Express Controller IP. Available online: <https://www.synopsys.com/designware-ip/interface-ip/pci-express.html> (accessed on 21 August 2025).
- Systems, C.D. Cadence PCI Express Controller IP. Available online: https://www.cadence.com/en_US/home/tools/ip/pcie.html (accessed on 21 August 2025).
- GitHub, Inc.; Forbes, K.; Xu, K.; Luszcz, J.; Tucker, M.; Brown, E.M.; Cihon, P.; Linksvayer, M.; Wolf, A.; Speiß, L.; et al. GitHub Open Source Survey 2024. 2024. Available online: <https://github.com/github/open-source-survey> (accessed on 26 March 2026).
- PCI-SIG. PCI Express Base Specification Revision 3.0. 2010. Available online: https://pcisig.com/PCIExpress/Specs/Base/_3.0 (accessed on 21 August 2025).
- Lange, J.; Ross, R. PCI Express in High-Performance Computing Systems. *Commun. ACM* **2021**, *64*, 12. [CrossRef]
- esynr3z. usb20dev: Open-Source USB 2.0 Device Core. Available online: <https://github.com/esynr3z/usb20dev> (accessed on 26 March 2026).
- Forencich, A. FPGA Taxi Documentation. Available online: <https://docs.fpga.taxi/en/latest/> (accessed on 26 March 2026).
- Hodgson, S.; Higgs, C. cocotb: Coroutine-Based Cosimulation Testbench for VHDL/Verilog. Potential Ventures; With Support from Solarflare Communications. Available online: <https://www.cocotb.org> (accessed on 21 August 2025).
- Kindgren, O. FuseSoC: Package Manager and Build System for HDL Code. 2024. Available online: <https://github.com/olofk/fusesoc> (accessed on 21 August 2025).
- Sharma, A. RISC-V 2024: A Year of Global Growth and Innovation. Available online: <https://riscv.org/blog/risc-v-2024-a-year-of-global-growth-and-innovation/> (accessed on 28 March 2026).
- Understanding PCIe Performance for End Host Networking. University of Cambridge Repository. Available online: <https://www.repository.cam.ac.uk/bitstreams/34b0b4ff-e448-41de-845b-07d73025542e/download> (accessed on 26 March 2026).

16. EnjoyDigital. LitePCIe: Open-Source PCIe Core. 2023. Available online: <https://github.com/enjoy-digital/litepcie> (accessed on 13 September 2025).
17. PCIe5 PHY Open Source Project. Focuses on Low-Level PCIe Gen5 PHY Modeling; Primarily Verification-Oriented. Available online: https://github.com/mgmtm98/pcie5_phy (accessed on 20 March 2026).
18. Hukare, S.; Vyas, V.; Agrawal, V. Design and Simulation of Physical Layer of Peripheral Component Interconnect Express (PCIe) Protocol. In Proceedings of the 2023 4th International Conference for Emerging Technology (INCET), Belgaum, India, 26–28 May 2023; pp. 1–4. [\[CrossRef\]](#)
19. Hyun, E.; Seong, K.S. The effective buffer architecture for data link layer of PCI express. In Proceedings of the International Conference on Information Technology: Coding and Computing, 2004. Proceedings. ITCC 2004, Las Vegas, NV, USA, 5–7 April 2004; Volume 1, pp. 809–813. [\[CrossRef\]](#)
20. Aguilar, M.; Veloz, A.; Guzman, M. Proposal of implementation of the “data link layer” of PCI-express. In Proceedings of the (ICEEE). 1st International Conference on Electrical and Electronics Engineering, 2004, Acapulco, Mexico, 8–10 September 2004; pp. 64–69. [\[CrossRef\]](#)
21. OpenCores PCI Express Project. Provides Partial Transaction-Level PCIe Implementations. Available online: <https://opencores.org/projects/pcie> (accessed on 13 September 2025).
22. openPCIE, 2025. Project Page. Available online: <https://github.com/chili-chips-ba/openPCIE/tree/main> (accessed on 28 March 2026).
23. pyuvm Contributors. pyuvm: Python UVM for Cocotb. 2023. Available online: <https://github.com/pyuvm/pyuvm> (accessed on 28 March 2026).
24. Gadde, D.N.; Kumari, S.; Kumar, A. Towards Efficient Design Verification—Constrained Random Verification using PyUVM. *arXiv* **2024**, arXiv:2407.10317. [\[CrossRef\]](#)
25. Arm Ltd. *AMBA AXI and ACE Protocol Specification (AXI4-Stream)*; Arm Ltd.: Cambridge, UK, 2021.
26. PCI-SIG. PHY Interface for PCI Express*, SATA, and USB 3.1: Specification. Available online: <https://www.intel.com/content/www/us/en/io/pci-express/phy-interface-pci-express-sata-usb30-architectures-3-1.html> (accessed on 28 March 2026).
27. Kelziye, B. PCIe_Scrambler: Verilog Implementation for PCIe Gen1/2/3 Scrambler Suitable for PIPE. 2025. Available online: https://github.com/baselkelziye/PCIe_Scrambler (accessed on 13 September 2025).
28. Initiative, A.S. *SystemRDL 2.0—Register Description Language*; Accellera Standard: Elk Grove, CA, USA, 2018.
29. Contributors, S. PeakRDL: SystemRDL Compilation Toolchain. 2025. Available online: <https://github.com/SystemRDL/PeakRDL> (accessed on 1 October 2025).
30. Jacobs, M.; Owen, A.; Nelson, J. A Systematic UVM-Based Verification Environment for PCIe Gen5 and DDR5 Interfaces. In Proceedings of the 2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT), Kamand, India, 24–28 June 2024.
31. Forencich, A. Cocotbext-Pcie: PCI Express Simulation Framework for Cocotb. 2025. Available online: <https://github.com/alexforencich/cocotbext-pcie> (accessed on 1 October 2025).
32. International Test Instruments Co., Ltd. 2500A PCI Express Protocol Analyzer. Available online: <http://www.internationaltestinstruments.com/products/100-itic-2500a-x4-25-gbps-pci-express-protocol-analyzer.aspx#> (accessed on 27 February 2026).
33. Asanović, K.; Avizienis, R.; Bachrach, J.; Beamer, S.; Biancolin, D.; Celio, C.; Cook, H.; Dabbelt, D.; Hauser, J.; Izraelevitz, A.; et al. The Rocket Chip Generator; Technical Report UCB/EECS-2016-17. 2016. Available online: <https://github.com/chipsalliance/rocket-chip> (accessed on 28 March 2026).
34. Parisi, E.; Musa, A.; Ciani, M.; Barchi, F.; Rossi, D.; Bartolini, A.; Acquaviva, A. Assessing the Performance of OpenTitan as Cryptographic Accelerator in Secure Open-Hardware System-on-Chips. In Proceedings of the P21st ACM International Conference on Computing Frontiers, New York, NY, USA, 7–9 May 2024; CF '24, pp. 172–179. [\[CrossRef\]](#)
35. Silva, A.I.; Susin, A.; Kastensmidt, F.L.; Beck, A.C.S.; Azambuja, J.R. NoX: A Compact Open-Source RISC-V Processor for Multi-Processor Systems-on-Chip. *arXiv* **2024**, arXiv:2406.17878. [\[CrossRef\]](#)
36. Perotti, M.; Cavalcante, M.; Wistoff, N.; Andri, R.; Cavigelli, L.; Benini, L. A “New Ara” for Vector Computing: An Open Source Highly Efficient RISC-V V 1.0 Vector Processor Design. In Proceedings of the 2022 IEEE 33rd International Conference on Application-Specific Systems, Architectures and Processors (ASAP); IEEE: New York, NY, USA, 2022; pp. 43–51. [\[CrossRef\]](#)
37. Liang, H.; Tan, N.; Ren, Y.; Hu, W.; He, J.; Xia, J. Python Based Testbench for Coverage Driven Functional Verification. In Proceedings of the 2022 7th International Conference on Integrated Circuits and Microsystems (ICICM), Xi’an, China, 28–31 October 2022; pp. 361–365. [\[CrossRef\]](#)
38. Bonvoisin, J.; Molloy, J.; Häuer, M.; Wenzel, T. Standardisation of Practices in Open Source Hardware. *J. Open Hardw.* **2020**, *4*, 4. [\[CrossRef\]](#)

39. Moritz, M.; Redlich, T.; Wulfsberg, J. Best Practices and Pitfalls in Open Source Hardware. In *Proceedings of the International Conference on Information Technology & Systems (ICITS 2018)*; Rocha, Á., Guarda, T., Eds.; Springer: Cham, Switzerland, 2018; pp. 200–210.
40. Parthasarathy, G.; Rushdi, A.; Choudhary, P.; Nanda, S.; Evans, M.; Gunasekara, H.; Rajakumar, S. RTL Regression Test Selection using Machine Learning. In *Proceedings of the 2022 27th Asia and South Pacific Design Automation Conference (ASP-DAC), Virtual*, 17–20 January 2022; pp. 281–287. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.